

ЛАБОРАТОРНА РОБОТА №3

Тема: Конструктори та деструктори в C++

Варіант 6

Мета роботи

Ознайомитися з принципами створення та використання конструкторів і деструкторів у мові програмування C++. Навчитися створювати декілька конструкторів для одного класу, використовувати конструктор за замовчуванням, конструктор з параметрами та конструктор копіювання, а також реалізовувати деструктор. Закріпити навички роботи з класами та об'єктами на основі класу `Angle`, створеного в лабораторній роботі №2.

Завдання

Використовуючи клас `Angle` з лабораторної роботи №2, реалізувати не менше трьох конструкторів.

Для класу `Angle` реалізувати:

1. Конструктор за замовчуванням.
2. Конструктор з двома параметрами — градусами та хвилинами.
3. Конструктор копіювання.
4. Деструктор.
5. Метод `Init()`.
6. Метод `Read()`.
7. Метод `Display()`.
8. Метод `toString()`.
9. Переведення кута в радіани.
10. Нормалізацію кута до діапазону від 0° до 360° .
11. Збільшення кута.
12. Зменшення кута.
13. Обчислення синуса кута.
14. Порівняння двох кутів.

Теоретичні відомості

Конструктор — спеціальний метод класу, який автоматично викликається під час створення об'єкта. Його основне призначення — початкова ініціалізація полів об'єкта.

Конструктор має таке саме ім'я, як і клас, та не має типу результату.

У C++ можна створити декілька конструкторів з різними наборами параметрів. Це називається перевантаженням конструкторів.

У роботі реалізовано три конструктори:

- конструктор за замовчуванням;
- конструктор з параметрами;
- конструктор копіювання.

Деструктор — спеціальний метод класу, який автоматично викликається під час знищення об'єкта. Назва деструктора складається з назви класу з символом `~` перед нею.

Для класу Angle використовуються два закритих поля:

- degrees — градуси;
- minutes — хвилини.

Один градус містить 60 хвилин.

Програмний код

```
#include <iostream>
#include <cmath>
#include <string>
#include <sstream>
#include <iomanip>
#include <locale>
```

```
using namespace std;
```

```
class Angle
```

```
{
```

```
private:
```

```
    int degrees;
```

```
    int minutes;
```

```
    int totalMinutes() const
```

```
    {
```

```
        return degrees * 60 + minutes;
```

```
    }
```

```
    void normalize()
```

```
    {
```

```
        int total = totalMinutes();
```

```
        const int fullCircle = 360 * 60;
```

```
        total %= fullCircle;
```

```
        if (total < 0)
```

```
            total += fullCircle;
```

```
        degrees = total / 60;
```

```
        minutes = total % 60;
```

```
    }
```

```
public:
```

```
    // 1. Конструктор за замовчуванням
    Angle()
```

```
{
    degrees = 0;
    minutes = 0;

    cout << "Викликано конструктор за замовчуванням."
          << endl;
}
```

// 2. Конструктор з параметрами

Angle(int d, int m)

```
{
    degrees = d;
    minutes = m;

    normalize();

    cout << "Викликано конструктор з параметрами."
          << endl;
}
```

// 3. Конструктор копіювання

Angle(const Angle& other)

```
{
    degrees = other.degrees;
    minutes = other.minutes;

    cout << "Викликано конструктор копіювання."
          << endl;
}
```

// Деструктор

~Angle()

```
{
    cout << "Викликано деструктор Angle."
          << endl;
}
```

// Ініціалізація об'єкта

void Init(int d = 0, int m = 0)

```
{
    degrees = d;
    minutes = m;
}
```

```

    normalize();
}

// Введення даних
void Read()
{
    cout << "Введіть градуси: ";
    cin >> degrees;

    cout << "Введіть хвилини: ";
    cin >> minutes;

    normalize();
}

// Виведення даних
void Display() const
{
    cout << degrees << "° "
        << minutes << "'" << endl;
}

// Перетворення в рядок
string toString() const
{
    ostringstream out;

    out << degrees << "° "
        << minutes << "'";

    return out.str();
}

// Переведення у радіани
double toRadians() const
{
    const double PI = 3.14159265358979323846;

    double angleInDegrees =
        degrees + minutes / 60.0;

    return angleInDegrees * PI / 180.0;
}

```

```

// Нормалізація кута
void normalizeAngle()
{
    normalize();
}

// Збільшення кута
void increase(int d, int m)
{
    int total =
        totalMinutes() + d * 60 + m;

    degrees = total / 60;
    minutes = total % 60;

    normalize();
}

// Зменшення кута
void decrease(int d, int m)
{
    int total =
        totalMinutes() - d * 60 - m;

    degrees = total / 60;
    minutes = total % 60;

    normalize();
}

// Обчислення синуса
double sine() const
{
    return sin(toRadians());
}

// Порівняння двох кутів
int compare(const Angle& other) const
{
    int thisAngle = totalMinutes();
    int otherAngle = other.totalMinutes();

```

```

        if (thisAngle < otherAngle)
            return -1;

        if (thisAngle > otherAngle)
            return 1;

        return 0;
    }

    // Оператор ==
    bool operator==(const Angle& other) const
    {
        return compare(other) == 0;
    }

    // Оператор <
    bool operator<(const Angle& other) const
    {
        return compare(other) < 0;
    }

    // Оператор >
    bool operator>(const Angle& other) const
    {
        return compare(other) > 0;
    }
};

int main()
{
    setlocale(LC_ALL, "");

    cout << "=== Перевірка конструктора за замовчуванням ==="
        << endl;

    Angle angle1;

    cout << "angle1 = ";
    angle1.Display();

    cout << endl;

    cout << "=== Перевірка конструктора з параметрами ==="

```

```
<< endl;

Angle angle2(120, 30);

cout << "angle2 = ";
angle2.Display();

cout << endl;

cout << "=== Перевірка конструктора копіювання ==="
    << endl;

Angle angle3(angle2);

cout << "angle3 = ";
angle3.Display();

cout << endl;

cout << "=== Введення першого кута ==="
    << endl;

angle1.Read();

cout << endl;

cout << "Перший кут: ";
angle1.Display();

cout << fixed << setprecision(6);

cout << "Радіани: "
    << angle1.toRadians()
    << endl;

cout << "Синус: "
    << angle1.sine()
    << endl;

cout << endl;

cout << "=== Операції з першим кутом ==="
    << endl;
```

```
angle1.increase(10, 30);

cout << "Після збільшення на 10° 30': ";
angle1.Display();

angle1.decrease(5, 15);

cout << "Після зменшення на 5° 15': ";
angle1.Display();

cout << endl;

cout << "=== Введення другого кута ==="
    << endl;

Angle angle4;

angle4.Read();

cout << "Другий кут: ";
angle4.Display();

cout << endl;

cout << "=== Порівняння кутів ==="
    << endl;

if (angle1 == angle4)
{
    cout << "Кути рівні." << endl;
}
else if (angle1 > angle4)
{
    cout << "Перший кут більший за другий."
        << endl;
}
else
{
    cout << "Перший кут менший за другий."
        << endl;
}
```

```
cout << endl;
```

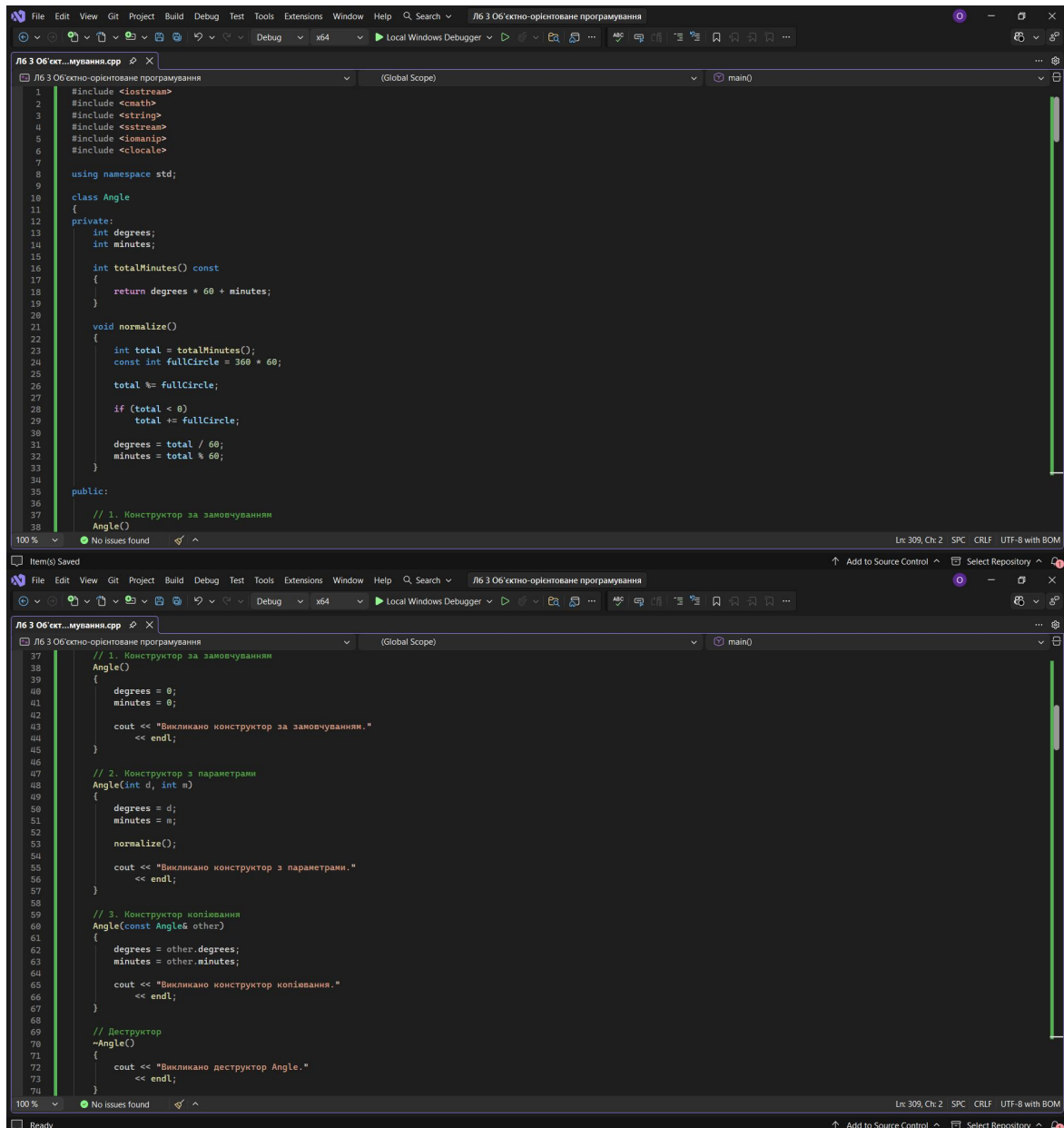
```
cout << "Результат toString() для першого кута: "  
    << angle1.toString()  
    << endl;
```

```
cout << endl;
```

```
cout << "=== Завершення роботи програми ==="  
    << endl;
```

```
return 0;
```

```
}
```

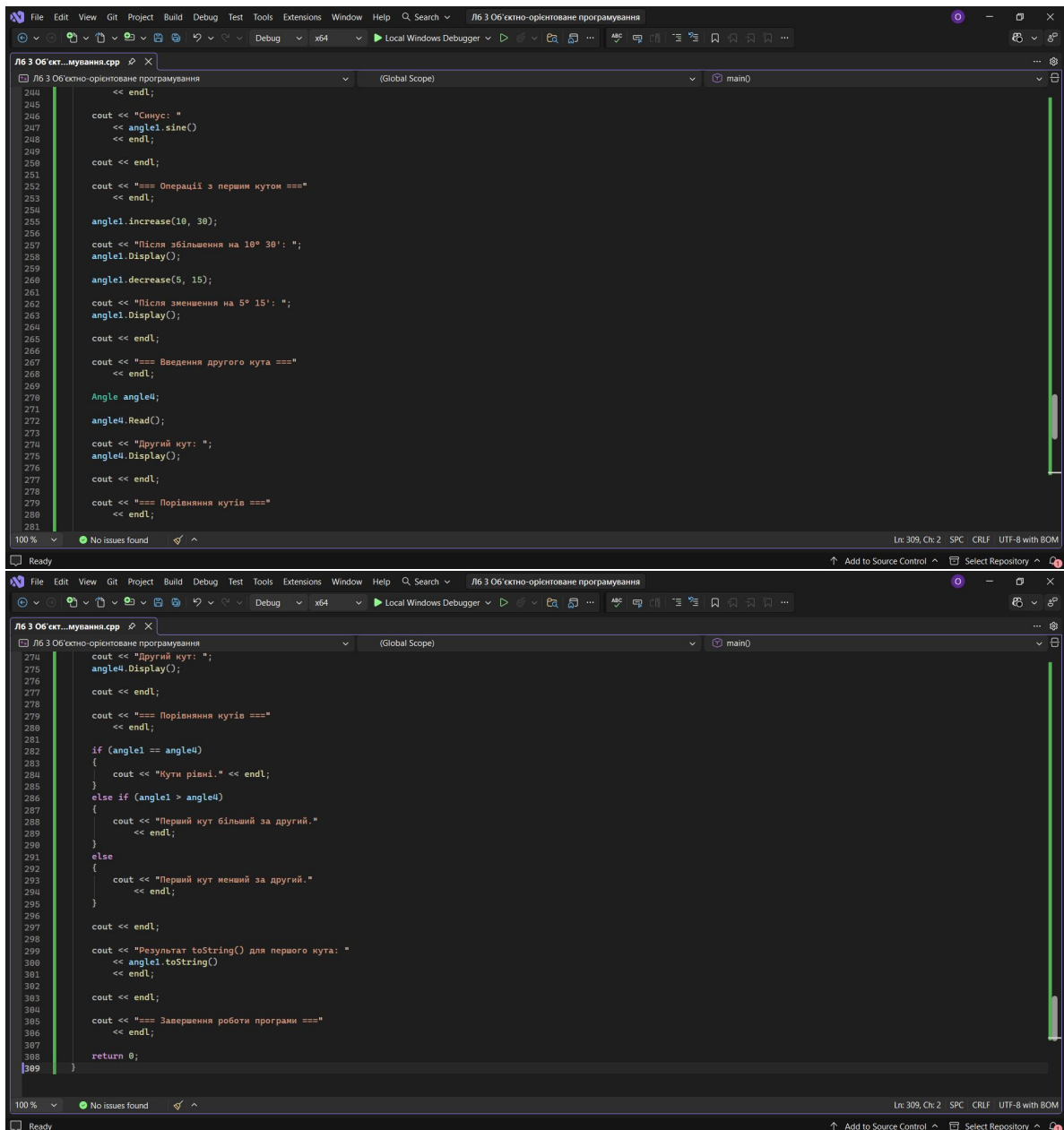


```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search /Л6 3 Об'єктно-орієнтоване програмування
Debug x64 Local Windows Debugger
Л6 3 Об'єкт...мування.cpp
Л6 3 Об'єктно-орієнтоване програмування (Global Scope) main()
73 }
74
75 // Ініціалізація об'єкта
76 void Init(int d = 0, int m = 0)
77 {
78     degrees = d;
79     minutes = m;
80     normalize();
81 }
82
83 // Введення даних
84 void Read()
85 {
86     cout << "Введіть градуси: ";
87     cin >> degrees;
88
89     cout << "Введіть хвилини: ";
90     cin >> minutes;
91     normalize();
92 }
93
94 // Виведення даних
95 void Display() const
96 {
97     cout << degrees << "° "
98         << minutes << "' << endl;
99 }
100
101 // Перетворення в рядок
102 string ToString() const
103 {
104     ostringstream out;
105     out << degrees << "° "
106         << minutes << "'";
107 }
108
109 100% No issues found Lnx 309, Ch: 2 SPC CRLF UTF-8 with BOM
```

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search /Л6 3 Об'єктно-орієнтоване програмування
Debug x64 Local Windows Debugger
Л6 3 Об'єкт...мування.cpp
Л6 3 Об'єктно-орієнтоване програмування (Global Scope) main()
109 out << degrees << "° "
110     << minutes << "'";
111 return out.str();
112 }
113
114 // Перетворення у радіани
115 double toRadians() const
116 {
117     const double PI = 3.14159265358979323846;
118
119     double angleInDegrees =
120         degrees + minutes / 60.0;
121     return angleInDegrees * PI / 180.0;
122 }
123
124 // Нормалізація кута
125 void normalizeAngle()
126 {
127     normalize();
128 }
129
130 // Збільшення кута
131 void increase(int d, int m)
132 {
133     int total =
134         totalMinutes() + d * 60 + m;
135     degrees = total / 60;
136     minutes = total % 60;
137     normalize();
138 }
139
140 // Зменшення кута
141 void decrease(int d, int m)
142 {
143 }
144
145 100% No issues found Lnx 309, Ch: 2 SPC CRLF UTF-8 with BOM
```

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search /Л6 3 Об'єктно-орієнтоване програмування
Debug x64 Local Windows Debugger
Л6 3 Об'єкт...мування.cpp
Л6 3 Об'єктно-орієнтоване програмування (Global Scope) main()
142 }
143
144 // Зменшення кута
145 void decrease(int d, int m)
146 {
147     int total =
148         totalMinutes() - d * 60 - m;
149     degrees = total / 60;
150     minutes = total % 60;
151     normalize();
152 }
153
154 // Обчислення синуса
155 double sine() const
156 {
157     return sin(toRadians());
158 }
159
160 // Порівняння двох кутів
161 int compare(const Angle& other) const
162 {
163     int thisAngle = totalMinutes();
164     int otherAngle = other.totalMinutes();
165     if (thisAngle < otherAngle)
166         return -1;
167     if (thisAngle > otherAngle)
168         return 1;
169     return 0;
170 }
171
172 // Оператор ==
173 bool operator==(const Angle& other) const
174 {
175 }
176
177 100% No issues found Lnx 309, Ch: 2 SPC CRLF UTF-8 with BOM
```

```
178     bool operator==(const Angle& other) const
179     {
180         return compare(other) == 0;
181     }
182
183     // Оператор <
184     bool operator<(const Angle& other) const
185     {
186         return compare(other) < 0;
187     }
188
189     // Оператор >
190     bool operator>(const Angle& other) const
191     {
192         return compare(other) > 0;
193     }
194 };
195
196 int main()
197 {
198     setlocale(LC_ALL, "");
199
200     cout << "=== Перевірка конструктора за замовчуванням ==="
201     << endl;
202
203     Angle angle1;
204
205     cout << "angle1 = ";
206     angle1.Display();
207
208     cout << endl;
209
210     cout << "=== Перевірка конструктора з параметрами ==="
211     << endl;
212
213     Angle angle2(120, 30);
214
215     cout << "angle2 = ";
216
217     Angle angle2(120, 30);
218     cout << "angle2 = ";
219     angle2.Display();
220
221     cout << endl;
222     cout << "=== Перевірка конструктора копіювання ==="
223     << endl;
224
225     Angle angle3(angle2);
226     cout << "angle3 = ";
227     angle3.Display();
228
229     cout << endl;
230     cout << "=== Введення першого кута ==="
231     << endl;
232
233     angle1.Read();
234
235     cout << endl;
236
237     cout << "Перший кут: ";
238     angle1.Display();
239
240     cout << fixed << setprecision(6);
241
242     cout << "Радіани: "
243         << angle1.toRadians()
244         << endl;
245
246     cout << "Синус: "
247         << angle1.sine()
248         << endl;
```



Пояснення конструкторів

1. Конструктор за замовчуванням

Angle()

```
{
    degrees = 0;
    minutes = 0;
}
```

Цей конструктор не має параметрів.

Він автоматично викликається під час створення об'єкта:

Angle angle1;

У результаті поля об'єкта отримують початкові значення:

degrees = 0

minutes = 0

2. Конструктор з параметрами

Angle(int d, int m)

```
{  
    degrees = d;  
    minutes = m;  
    normalize();  
}
```

Цей конструктор дозволяє одразу задати значення кута.

Наприклад:

Angle angle2(120, 30);

Створюється кут:

120° 30'

Після встановлення значень викликається normalize(), тому кут автоматично приводиться до допустимого діапазону.

3. Конструктор копіювання

Angle(const Angle& other)

```
{  
    degrees = other.degrees;  
    minutes = other.minutes;  
}
```

Конструктор копіювання створює новий об'єкт на основі вже існуючого.

Наприклад:

Angle angle2(120, 30);

Angle angle3(angle2);

Після цього:

angle2 = 120° 30'

angle3 = 120° 30'

Таким чином, значення полів angle2 копіюються до angle3.

Деструктор

У класі реалізовано деструктор:

~Angle()

```
{  
    cout << "Викликано деструктор Angle."
```

```
<< endl;  
}
```

Деструктор автоматично викликається під час знищення об'єкта.

У даній програмі об'єкти створюються у функції `main()`, тому деструктори викликаються автоматично після завершення роботи `main()`.

Перевірка конструктора за замовчуванням

Для перевірки створюється об'єкт:

```
Angle angle1;
```

Очікуваний результат:

```
=== Перевірка конструктора за замовчуванням ===
```

Викликано конструктор за замовчуванням.

```
angle1 = 0° 0'
```

Перевірка конструктора з параметрами

Створюється об'єкт:

```
Angle angle2(120, 30);
```

Результат:

```
=== Перевірка конструктора з параметрами ===
```

Викликано конструктор з параметрами.

```
angle2 = 120° 30'
```

Перевірка конструктора копіювання

Виконується:

```
Angle angle3(angle2);
```

Результат:

```
=== Перевірка конструктора копіювання ===
```

Викликано конструктор копіювання.

```
angle3 = 120° 30'
```

Отже, конструктор копіювання працює правильно.

Приклад виконання програми

```
=== Перевірка конструктора за замовчуванням ===
```

Викликано конструктор за замовчуванням.

```
angle1 = 0° 0'
```

```
=== Перевірка конструктора з параметрами ===
```

Викликано конструктор з параметрами.

```
angle2 = 120° 30'
```

=== Перевірка конструктора копіювання ===
Викликано конструктор копіювання.
 $\text{angle3} = 120^\circ 30'$

=== Введення першого кута ===
Введіть градуси: 575
Введіть хвилини: 2

Перший кут: $215^\circ 2'$
Радіани: 3.753040
Синус: -0.574053

=== Операції з першим кутом ===
Після збільшення на $10^\circ 30'$: $225^\circ 32'$
Після зменшення на $5^\circ 15'$: $220^\circ 17'$

=== Введення другого кута ===
Викликано конструктор за замовчуванням.
Введіть градуси: 53
Введіть хвилини: 5
Другий кут: $53^\circ 5'$

=== Порівняння кутів ===
Перший кут більший за другий.

Результат toString() для першого кута: $220^\circ 17'$

=== Завершення роботи програми ===
Викликано деструктор Angle.
Викликано деструктор Angle.
Викликано деструктор Angle.
Викликано деструктор Angle.

C:\Users\oleks\source\repos\Лб 3 Об'єктно-орієнтоване
програмування\x64\Debug\Лб 3 Об'єктно-орієнтоване програмування.exe
(process 39452) exited with code 0 (0x0).
Press any key to close this window . . .

```
Microsoft Visual Studio Debug x + v
=== Перевірка конструктора за замовчуванням ===
Викликано конструктор за замовчуванням.
angle1 = 0° 0'

=== Перевірка конструктора з параметрами ===
Викликано конструктор з параметрами.
angle2 = 120° 30'

=== Перевірка конструктора копіювання ===
Викликано конструктор копіювання.
angle3 = 120° 30'

=== Введення першого кута ===
Введіть градуси: 575
Введіть хвилини: 2

Перший кут: 215° 2'
Радіани: 3.753040
Синус: -0.574053

=== Операції з першим кутом ===
Після збільшення на 10° 30': 225° 32'
Після зменшення на 5° 15': 220° 17'

=== Введення другого кута ===
Викликано конструктор за замовчуванням.
Введіть градуси: 53
Введіть хвилини: 5
Другий кут: 53° 5'

Введіть хвилини: 2

Перший кут: 215° 2'
Радіани: 3.753040
Синус: -0.574053

=== Операції з першим кутом ===
Після збільшення на 10° 30': 225° 32'
Після зменшення на 5° 15': 220° 17'

=== Введення другого кута ===
Викликано конструктор за замовчуванням.
Введіть градуси: 53
Введіть хвилини: 5
Другий кут: 53° 5'

=== Порівняння кутів ===
Перший кут більший за другий.

Результат toString() для першого кута: 220° 17'

=== Завершення роботи програми ===
Викликано деструктор Angle.
Викликано деструктор Angle.
Викликано деструктор Angle.
Викликано деструктор Angle.

C:\Users\oleks\source\repos\Л6 3 Об'єктно-орієнтоване програмування\х64\Debug\Л6 3 Об'єктно-орієнтоване програмування.exe (process 39452) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Результати роботи

У програмі перевірено роботу трьох конструкторів:

1. Angle() — конструктор за замовчуванням.
2. Angle(int d, int m) — конструктор з параметрами.
3. Angle(const Angle& other) — конструктор копіювання.

Також перевірено роботу деструктора:

~Angle()

Після створення об'єктів було перевірено основні функції класу Angle: введення та виведення кута, нормалізацію, переведення у радіани, обчислення синуса, збільшення, зменшення та порівняння кутів.

Висновок

У ході виконання лабораторної роботи було вдосконалено клас `Angle`, створений у лабораторній роботі №2.

У класі реалізовано три конструктори: конструктор за замовчуванням, конструктор з параметрами та конструктор копіювання. Також було реалізовано деструктор, який автоматично викликається під час знищення об'єктів.

Було перевірено правильність створення об'єктів різними конструкторами та копіювання одного об'єкта в інший. Збережено функціональність класу `Angle`, зокрема введення і виведення даних, нормалізацію кута, переведення у радіани, обчислення синуса, збільшення та зменшення кута, а також порівняння двох кутів.

У результаті виконання лабораторної роботи було закріплено практичні навички роботи з конструкторами, конструктором копіювання та деструкторами в мові програмування `C++`.